An overview of NLP evaluation metrics and distance/similarity measures

Created	@May 15, 2020 12:21 PM
Updated	@April 13, 2022 4:01 PM
Status	Discarded

Introduction
Evaluation Metrics
- NLG Evaluations
  - 1. Bilingual Evaluation Understudy(BLEU)
  - 2. NIST
  - 3. Recall Oriented Understudy for Gisting Evaluation(ROUGE)
  - 4. Metric for Evaluation for Translation with Explicit Ordering(METEOR)
  - 5. Translation Error Rate(TER)/Word Error Rate(WER)
- More Seq-Seq Evaluations
  - 6. Subtree Metric(STM)
  - 7. hLEPOR
  - 8. Machine Translation Evaluation without Reference Texts(MEWR)

Distance/Similarity Measures
- 1. Jaccard Similarity/Jaccard Index/Intersection over Union
- 2. Edit Distance/Levenshtein Distance
- 3. Hamming Distance/Signal Distance
- 4. Cosine Similarity
- 5. Soft Cosine Similarity
- 6. Euclidean Distance/L2 Norm
- 7. Manhattan Distance/Taxicab Distabce/City Block Distance/L1 Norm
- 8. Minkowski Distance/P Norm
- 9. Word Mover's Distance (WMD)
- 10. Term Frequency-Inverse Document Frequency(TF-IDF)

Resources

Introduction

Evaluation in NLP is an open problem especially when it comes to the evaluation of generated text. It's a really hard problem and therefore there is no single go to evaluation metric available. What makes this problem even more hard is that metrics need to evaluate competing goals i.e Correctness (quality)/Specicity (diversity).

How can you evaluate a model.

1. Indirect-Evaluation - Incorporate your NLP model into some other downstream task that can be easily evaluaed.
2. Direct Evaluation
    a. Human Evaluation
    b. Automatic Evaluation - Generate some objective scores using ground truths and an evalation metric.

Then the question is which metric to use in which case. And the best way to answer this questions is to first answer the question - what are we trying to measure?

Evaluation Metrics

NLG Evaluations

In natural language generation tasks such as translation, summarization etc, a generated candidate A is considered better than the candidate B if the matching score of that candidate with respect to the ground truth candidate is more than the other. Most of the metrics in their definition try to capture the syntactic similarity by comparing the precision/recall of various candidates.

1. Bilingual Evaluation Understudy(BLEU)

It was originally developed to measure the quality of machine translation. The metric uses a modified version of n-gram precision for computing the BLEU score. An n-gram precision is defined as the fraction of n-grams in the generated text which are present in any of the reference(ground truth) texts. But this definition of n-gram precision is not enough to capture the following case.

> Reference text : the cat is on the mat
> Generated text1 : cat cat cat cat cat cat
> Generated text2 : the cat is on the mat
> Generated text3 : the mat on is the cat
> Generated text4 : mat

All the generated texts have a precison of 1 since all the generated unigrams are also present in the reference text. But clearly apart from the second generated text rest all are meaningless and they should be scored very low in comparison to the second generated text. To solve this problem, BLEU uses a modifed n-gram precison where it matches the generated text's n-grams only as many times as they are present in any of the reference texts. So in the above example, cat is matched only once and hence the unigram precision for generated text1 will be 1/6.

But this doesn't solve the problem with the third generated text which still has the modified n-gram precision of 1. To overcome this, instead of just relying on unigrams BLEU scores are based on an average of unigram, bigram, trigram and 4-gram precision.

The above two workarounds will not solve the problem with the genrated text4 it will still have an n-gram precision of 1. One way of getting around this is by multiplying the score we have so far by a measure that penalizes sentences that are shorter than any of our reference texts. This is the **brevity penalty**.

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

$$\text{Brevity Penalty} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

> BP : Brevity penalty
> r : count of words in a reference translation
> c : count of words in a candidate translation
> N: No. of n-grams, BLEU usually use unigrams, bigrams, trigrams, 4-grams
> w_n: Weight constant for each modified precision, w_n is 1/N.
> P_n: Modified precision

The BLEU metric ranges from 0 to 1 and higher is better.

Some important things to keep in mind while using BLEU.

1. BLEU measures the closeness of the generated text to reference texts taking **generation length, word choice, and word order** into consideration.

2. It’s **fast and easy to calculate**, especially compared to having human evaluators.
3. BLEU was always intended to be a corpus-level measure. Taking the BLEU score of each sentence in the corpus and then averaging across them will artificially inflate your score.
4. It doesn’t consider meaning. It means that a **difference in a stop word (like “an” or “on”) is penalized as heavily as a difference in a more important content word.** This is handled by NIST.
5. It **doesn’t handle morphologically rich languages well** since it is based on word-level n-grams and not character-level n-grams.

2. NIST

Its name comes from the US National Institute of Standards and Technology. It is **based on the BLEU** metric, but with some modifications. Where BLEU simply calculates n-gram precision adding equal weight to each one, NIST also calculates how informative a particular n-gram is. That is to say when a correct n-gram is found, **the rarer that n-gram is, the more weight it will be given**. For example, if the bigram "on the" is correctly matched, it will receive lower weight than the correct matching of bigram "interesting calculations", as this is less likely to occur.

3. Recall Oriented Understudy for Gisting Evaluation(ROUGE)

ROUGE is **based only on recall instead of precision**. ROUGE may be of many types, namely ROUGE-N, ROUGE-L(measures the longest common subsequence (LCS) between our model output and reference.) and ROUGE-W etc. ROUGE-N is based on n-grams. For example, ROUGE-1 counts recall based on matching unigrams, and so on. For any n, we count the total number of n-grams across all the reference texts, and find out how many of them are present in the generated text. This fraction is the required metric value.

4. Metric for Evaluation for Translation with Explicit Ordering(METEOR)

It's another metric that was originally developed for evaluating machine translation. The metric was designed to fix some of the problems found in the more popular BLEU metric, and also produce **good correlation with human judgement** at the sentence or segment level. This differs from the BLEU metric in that BLEU seeks correlation at the corpus level. To solve this problem, METEOR modifies the precision and recall computations, **replacing them with a weighted F-score** where recall has a heigher weight. It also has several features that are not found in other metrics, such as **stemming and synonym matching**.

Some important things to keep in mind while using METEOR

1. It produces better correlation with human judgement at the sentence compared to BLEU.
2. It also uses stemming and synonym matching for defining a unigram match. Because of the stemming it works **better with morphologically** rich languages than BLEU score.

5. Translation Error Rate(TER)/Word Error Rate(WER)

It measures the number of edits needed to chance the original output translation into an acceptable human-level translation. TER is defined as the minimum number of edits needed to change a hypothesis so that it exactly matches one of the references, normalized by the average length of the references. Since we are concerned with the minimum number of edits needed to modify the hypothesis, we only measure the number of edits to the closest reference (as measured by the TER score). Specifically:

$$TER = \text{number of edits}/\text{average number of reference words}$$

Possible edits include the insertion, deletion, and substitution of single words as well as shifts of word sequences. A shift moves a contiguous sequence of words within the hypothesis to another location within the hypothesis. All edits, including shifts of any number of words, by any distance, have equal cost.

There are enhanced versions of TER that also considers paraphrases, stemming, and synonyms.

More Seq-Seq Evaluations

All the above metrics were mainly based on the idea that the closer a machine translation is to a professional human(referece) translation, the better it is. This type of feature cannot capture the grammaticality of the sentence, in part because they do not take into account sentence-level information. For example, a sentence can achieve an excellent BLEU score without containing a verb. Consider the following simple example:

> Reference: I had a dog.
> Hypothesis 1: I have the dog.
> Hypothesis 2: A dog I had.

If we use BLEU to evaluate these two sentences, hypothesis 2 has two bigrams a dog and I had which are also found in the reference, and hypothesis 1 has no bigrams in common with the reference. Thus hypothesis 2 will get a higher score than hypothesis 1. The result is obviously incorrect.

6. Subtree Metric(STM)

It uses the **syntax tree of a languge as a feature to measure the similarity between a generated sentence and a reference sentence**. When this approach is applied on the above example we get our desired result of hypothesis 1 being better than hypothesis 2.

$$\mathrm{STM} = \frac{1}{D} \sum_{n=1}^{D} \frac{\sum_{t \in \mathrm{subtrees}_n(hyp)} \mathrm{count}_{\mathrm{clip}}(t)}{\sum_{t \in \mathrm{subtrees}_n(hyp)} \mathrm{count}(t)}$$

where D is the maximum depth of subtrees considered, count(t) denotes the number of times subtree t appears in the candidate's syntax tree, and countclip(t) denotes the clipped number of times t appears in the references' syntax trees. Clipped here means that, for a given subtree, the count computed from the hypothesis syntax tree can not exceed the maximum number of times the subtree occurs in any single reference's syntax tree.

7. hLEPOR

Some metrics perform well on certain languages but weak on other languages, which is usually called as language bias problem. To make this metric perform well on different language pairs, extensive factors are designed and the assigned **parameter weights which are tunable according to the special characteristics of focused language** pairs. **Among other factors similar to BLEU it also considers things like part-of-speech (noun, verb, etc.)** that can help capture syntactic information.

8. Machine Translation Evaluation without Reference Texts(MEWR)

Automatic metrics such as BLEU etc requrire reference texts, however for many applications **reference texts are not available or are expensive to generate**. MEWR is an evaluation metric for machine translation that **doesn't require reference texts**. It uses a combination of word and sentence embeddings (which capture some aspects of meaning) and perplexity to score translations.

1. It first tries to learn a **transformation matrix between the word embeddings of the source and target language.** For this, they learn a matrix W that minimize the

euclidean distance between Wxi and zi where xi and zi are the word embeddings in source and target language for the ith word.

2. Using word embedding **transformation matrix W it maps source sentence embeddings to the target sentence embeddings.** Then, it computes the consine similarity between the target sentence embeddings generated by this transformation and the embedding of the actual translated sentence generated by the translation model.
3. It uses **perplexity for measuring the fluency** of the generated text.

Distance/Similarity Measures

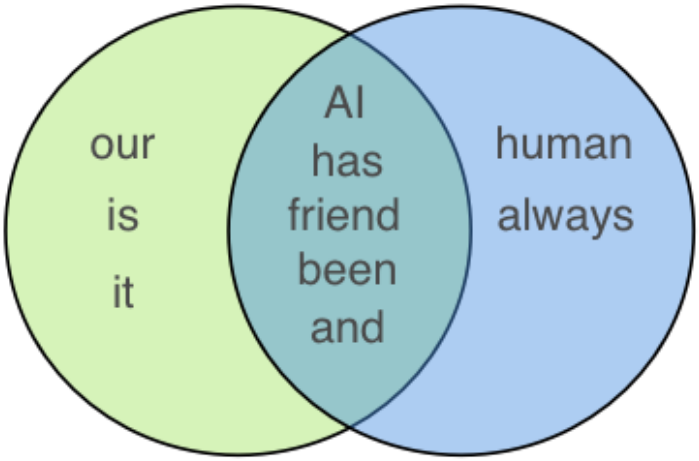
1. Jaccard Similarity/Jaccard Index/Intersection over Union

It measures similarity/diveristy between two sample sets. It is simply defined as the **size of the intersection divided by the size of the union** of the sample sets.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

There's no limit on how you want to define the sets. Depending on the use case a set can either be defined as a set of words, or a set of ngrams or a set of characters.

> Sentence 1: AI is our friend and it has been friendly
> Sentence 2: AI and humans have always been friendly



For the above two sentences, we get Jaccard similarity of 5/(5+3+2) = 0.5 which is size of intersection of the set divided by total size of set.

Some important things to keep in mind while using Jaccard similarity.

1. This approach has an inherent flaw. That is, **as the size of the document increases, the number of common words tend to increase** even if the documents talk about different topics. One way to overcome this is to perform `stop word removal` and `lemmatization` before creating the sets.
2. Jaccard similarity is not able to capture `semantic similarity`. For instance, the following sentences will have a Jaccard similarity of 0 even though they have similar meaning.

   > Sentence 1: President greets the press in Chicago
   > Sentence 2: Obama speaks in Illinois

3. **The order of words doesn't impact the score**, so two really different sentences with similar words will have a high Jaccard similarity score. A workaround for this is to use **character ngrams** instead of words as set elements.
4. Due to the above reasons Jaccard similarity might be more useful for duplicate detection rather than similarity detection.

2. Edit Distance/Levenshtein Distance

Edit distance is a way of measuring how dissimilar two sequences are to one another by counting the minimum number of operations required to transform one sequence into

the other. The allowed operations are `removal`, `insertion`, or `substitution` of a character in the sequence.

1. Some common application of Edit distance in the field of NLP are - spelling correction, approximate string matching or fuzzy matching where a small number of differences are allowed.
2. Edit distance is much more intuitive, for instance, `abcdefg` and `bcdefgh` have an edit distance of 2 whereas their hamming distance is 7.
3. **It is computationallly expensive to calculate edit distance for long sequences**. Most Common algorithm using dynamic programming has a time and space complexity of Θ(mn) where m,n are the lengths of two sequences.

3. Hamming Distance/Signal Distance

It is one of the several metrics used to measure changes between two sequences of equal lengths. It is defined as the count of number of positions at which the corresponding characters are different. In other words, it measures the minimum number of `substitutions` required to change one sequence into the other.

Some important things to keep in mind while using Hamming distance

1. Unlike edit distance Hamming distance only allows substitution.
2. It is **considered too strict**, for example, `abcdefg` and `bcdefgh` are considered totally different (with hamming distance 7), while 6 out of 7 characters are the same.
3. Historically, it is used to count the number of flipped bits in a fixed-length binary word as an estimate of error.
4. One possible application of Hamming distance in NLP is to measure difference between two fixed-length one hot encoded vectors.
5. It can also be used as a distance metric in K-nearest neighbour classification and K-means clustering of one hot encoded categorical vectors.

4. Cosine Similarity

It is the measure of orientation and not magnitude. Mathematically, it calculates the cosine of the angle between two vectors. Vectors which are similar should be oriented similarly and hence will have smaller angle between them. The cosine of 0° is 1, and 90° is 0.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

It can also be defined as the `dot product` of two `unit vectors`.

1. One advantage of cosine similarity is that the scores are normalized. So even if the two similar vectors are far apart in terms of Euclidean distance due to scaling issues, they will still be oriented similarly and hence the cosine similarity will be high.
2. Some good reasons why consine similarity makes more sense than euclidean distance when comparing words can be read here and here.

5. Soft Cosine Similarity

Traditional cosine similarity considers features as independent of each other where soft cosine proposes to consider similarity between pairs of features as well. For instance, if we are computing document similarity by using one hot encoded vocabulary vectors, then one document might have the word play and other might have the word playing or game. Athough they might represent different words in the vocabulary but still they have some syntatic and semantic similarity. It might be useful to take this into account while computing the overall document similarity.

For calculating soft cosine, the matrix s is used to indicate similarity between features. **It can be calculated through Levenshtein distance or other similarity measures**. Then we just multiply by this matrix.

$$\text{soft_cosine}_1(a, b) = \frac{\sum_{i,j}^{N} s_{ij} a_i b_j}{\sqrt{\sum_{i,j}^{N} s_{ij} a_i a_j}\sqrt{\sum_{i,j}^{N} s_{ij} b_i b_j}},$$

where sij = similarity(featurei, featurej).

If there is no similarity between features (sii = 1, sij = 0 for i ≠ j), the given equation is equivalent to

the conventional cosine similarity formula.

6. Euclidean Distance/L2 Norm

It is the straight line distance between two points in an euclidean plane. It is generally computed for vectors consisting of features with integer or floating values. To reduce the impact of one partiular feature of the vector over the final distance value it is advised to first normalize the vectors.

$$d(\mathbf{p}, \mathbf{q}) = d(\mathbf{q}, \mathbf{p}) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \cdots + (q_n - p_n)^2}$$
$$= \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}.$$

Euclidean is a good distance measure to use if **all the features are of similar type** (e.g. all measure widths and heights). Essentially, range of values for all the features should be similar.

7. Manhattan Distance/Taxicab Distabce/City Block Distance/L1 Norm

$$d_1(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_1 = \sum_{i=1}^{n} |p_i - q_i|,$$

Manhattan distance is a good measure to use if all the feature values **are not of similar type** (such as age, gender, height, etc.). Essentially, when range of values for all the features are not similar.

8. Minkowski Distance/P Norm

It is considered as a generalization of both the Euclidean distance and the Manhattan distance. p≥1 is a hyperparameter in this case and can be tuned according to your usecase.

$$D(X, Y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

9. Word Mover’s Distance (WMD)

WMD was designed to overcome the problem of syntatic similarity. **In order to calculate the distance it makes use of word embeddings and relies on the assumption that words with similar meaning have similar word embeddings**.

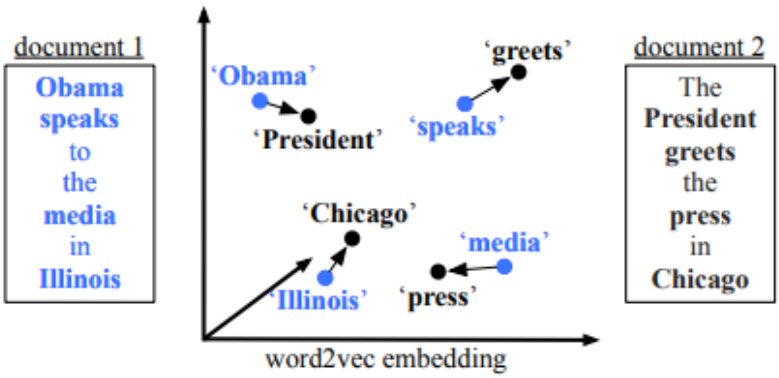


Figure 1. An illustration of the *word mover's distance*. All non-stop words (***bold***) of both documents are embedded into a *word2vec* space. The distance between the two documents is the minimum cumulative distance that all words in document 1 need to travel to exactly match document 2. (Best viewed in color.)

These two sentences consists of different words if we ignore the stop words. This means that they are syntactically very far apart. However, these sentences may still be semantically close. WMD computes the distance using the following steps.

1. **Remove stop words and create an n(vocabulary size) dimensional bag of words(nBOW) vector for words in the given sentences.** Let's denote these nBOW vectors as `d` and `d'`. The algorithm also assumes that the more frequent a word is the more important it is for the distance metric. They represent this using a normalized word frequency. If word `i` appears `ci` times in the document, we denote

$$d_i = \frac{c_i}{\sum_{j=1}^{n} c_j}$$

2. Now the goal is to incorporate semantic similarity between words into the sentence similarity metric. For this Euclidean distance(using a word embedding of you choice) is used to define dissimilarity between words. For instance, the distance between word `i` and word `j` becomes `c(i, j) = ||xi - xj||2` where xi and xj are the word embeddings for the i and j words. In other words, `c(i, j)` is the **cost associated with "traveling" from one word to another.**

3. **The algorithm doesn't know which word in one sentence should be transformed/transferred to which word in the other senetnce. So, the algorithm allow each word i in d to be transformed into any word in d' in total or in parts.** For this, the algorithm defines a transportation matrix T ∈ Rn×n where Tij ≥ 0 denotes how much of word i in d travels to word j in d'. This matrix just defines the propotion of the word flow and not the cost associated with it.

4. Entire outgoing flow from word i equals di. Further, the amount of incoming flow to word j must match d'j.

$$\sum_j T_{ij} = d_i$$

$$\sum_i T_{ij} = d'_j$$

5. Finally, the distance between the two documents is defined as the minimum (weighted) cumulative cost required to move all words from d to d'. The whole algorithm can be formulated as an optimiation problem and as a special case of the Earth Mover's distance metric(EMD).

$$\begin{aligned} \min_{\mathbf{T} \geq 0} \sum_{i,j=1}^{n} \mathbf{T}_{ij} c(i,j) \\ \text{subject to: } \sum_{j=1}^{n} \mathbf{T}_{ij} &= d_i \quad \forall i \in \{1, \ldots, n\} \\ \sum_{i=1}^{n} \mathbf{T}_{ij} &= d'_j \quad \forall j \in \{1, \ldots, n\}. \end{aligned}$$

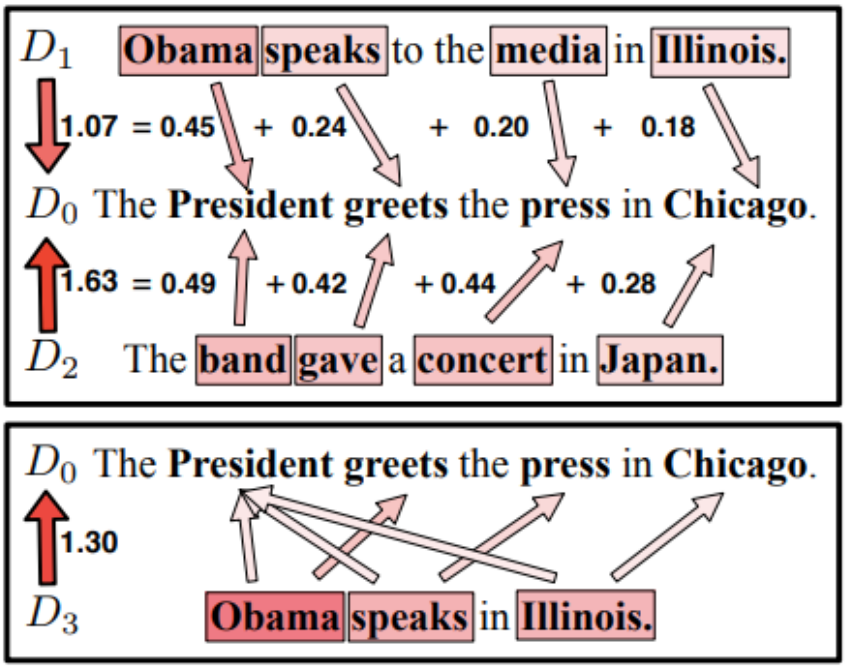


Figure 2. (Top:) The components of the WMD metric between a query D_0 and two sentences D_1, D_2 (with equal BOW distance). The arrows represent flow between two words and are labeled with their distance contribution. *(Bottom:)* The flow between two sentences D_3 and D_0 with different numbers of words. This mismatch causes the WMD to move words to multiple similar words.

Some important things to keep in mind while using WMD

1. The best average time of solving WMD is about O(p^3 log p) where p is number of unique words. There are some better versions such as Relaxed Word Mover's Distance(RWMD) with time complexity of O(p^2).
2. The advantage of WMD is that it is hyper-parameter free and it can overcome the synonym problem by sematically comparing words.
3. It does not consider word ordering.

10. Term Frequency-Inverse Document Frequency(TF-IDF)

It is a stastical measure that computes how relevant a word is to a document with respect to other documents. To calculate the TF-IDF for a word follow these steps.

1. **Term Frequency** - This measures the frequency of a word in a document. Documents can vary in size and some common words might appear more

frequently in longer documents. To overcome this we can perform a normalization of term frequency by dividing it with the total count of words in a document. The final value of the normalised TF value will be in the range of [0 , 1]. Formally it is defined as follows.

$$tf(t,d) = \text{count of t in d}/\text{number of words in d}$$

There's a problem with directly using `tf` for defining relevance between documents. Stop words such as `are`, `an`, `the` are very common in documents and so will have high `tf` values and hence comparing documents with just `tf` will produce in-accurate results.

2. **Document Frequency** - This measures the occurance of a word in all the documents of the corpus. We consider one occurrence if the word appears in the document at least once, we do not need to know the number of times the word is present. We also normalize this by dividing it with the total number of documents in the corpus.

$$df(t) = \text{occurrence of t in documents}/N$$

3. **Inverse Document Frequency** - This measures the informativeness(rarity) of a word in the corpus. By definition this value should be high for words which are rare and hence more informative. It is defined as the inverse of the `df`.

$$idf(t) = 1/df$$

When we calculate `idf`, it will be a small value(close to 1) for most stop words because these words are very common and are present in almost all of the documents. Another problem with current definition of `idf` is that **it explodes if the value of `N` becomes too large. So to dampen the effect it is advised to take the log** of `idf`. A word might not be present in a document and so it `df` will be 0. As we cannot define the `idf` in this case(because divide by 0), **we smoothen the value by adding 1 to the denominator**.

$$idf(t) = log(N/(df+1))$$

4. Finally, the basic version of TF-IDF is defined as

$$tfidf(t,d) = tf(t,d) * log(N/(df+1))$$

Some important things to keep in mind while using TF-IDF

1. Preprocessing steps such as converting to lowercase, removing punctuation, removing stop words and lemmatization/stemming are very important when using TF-IDF.
2. TF-IDF can also be used for domain based stop-words filtering.
3. Since TF-IDF is based on BOW, it can not capture word orderings, semantics etc.

Resources

1. https://people.cs.umass.edu/~brenocon/inlp2015/15-eval.pdf
2. https://drive.google.com/file/d/1P--z-jGWMtrSHdjB64X4BEwuIedzkO6N/view
3. https://towardsdatascience.com/evaluating-text-output-in-nlp-bleu-at-your-own-risk-e8609665a213
4. https://www.cs.colorado.edu/~jbg/teaching/CMSC_773_2012/reading/evaluation.pdf
5. https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/
6. https://thegradient.pub/understanding-evaluation-metrics-for-language-models/
7. https://medium.com/fintechexplained/nlp-how-to-evaluate-the-model-performance-7e1820cdb759
8. https://medium.com/@adriensieg/text-similarities-da019229c894
9. http://text2vec.org/similarity.html
10. https://towardsdatascience.com/3-text-distances-that-every-data-scientist-should-know-7fcdf850e510
11. https://machinelearningmastery.com/distance-measures-for-machine-learning/
12. https://stats.stackexchange.com/questions/341535/curse-of-dimensionality-does-cosine-similarity-work-better-and-if-so-why
13. https://www.quora.com/Why-cosine-is-better-than-Euclidean-in-high-dimensional-data-as-in-text-documents
14. https://machinelearningmastery.com/k-nearest-neighbors-for-machine-learning/
15. https://towardsdatascience.com/word-movers-distance-for-text-similarity-7492aeca71b0
16. https://towardsdatascience.com/word-distance-between-word-embeddings-cc3e9cf1d632

17. http://proceedings.mlr.press/v37/kusnerb15.pdf
18. https://towardsdatascience.com/tf-idf-for-document-ranking-from-scratch-in-python-on-real-world-dataset-796d339a4089
19. https://en.wikipedia.org/wiki/NIST_(metric)
20. https://www.aclweb.org/anthology/P02-1040.pdf
21. https://towardsdatascience.com/bleu-bilingual-evaluation-understudy-2b4eab9bcfd1
22. https://medium.com/explorations-in-language-and-learning/metrics-for-nlg-evaluation-c89b6a781054
23. https://www.aclweb.org/anthology/W05-0904.pdf
24. http://www.mt-archive.info/10/MTS-2013-Han.pdf#
25. https://pbs.twimg.com/media/DQJBTbxV4AApFJE.jpg:large
26. http://www.cs.umd.edu/~snover/pub/amta06/ter_amta.pdf